EXPERIMENT - 7

Normalization of the Sales Relation

AIM

To identify functional dependencies in a denormalized Sales table and normalize the relation step-by-step from UNF to 1NF, 2NF, 3NF and BCNF, and to verify lossless join and dependency preservation using SQL queries.

1. CREATE UNNORMALIZED SALES TABLE

```
CREATE DATABASE IF NOT EXISTS Experiment7;
USE Experiment7;
DROP TABLE IF EXISTS Sales_UNF;
CREATE TABLE Sales_UNF (
 Sale_ID INT PRIMARY KEY, Sale_Date DATE, Customer_ID INT,
 Customer_Name VARCHAR(50), Customer_City VARCHAR(50),
 Salesperson_ID INT, Salesperson_Name VARCHAR(50),
 Product_IDs VARCHAR(100), Product_Names VARCHAR(255),
 Quantities VARCHAR(100), Payment_Mode VARCHAR(20)
);
INSERT INTO Sales_UNF VALUES
(1001,'2026-10-01',101,'Aarav Sharma','Delhi',501,'Neha Gupta','P01,P02','Laptop,Mouse','1,2','UPI'),
(1002,'2026-10-02',102,'Riya Verma','Lucknow',502,'Rahul Singh','P02,P03','Mouse,Keyboard','1,1','CARD'),
(1003,'2026-10-03',103,'Karan Mehta','Kanpur',501,'Neha Gupta','P01,P04','Laptop,Headphones','1,1',
'CASH');
```

UNF contains repeating/multi-valued product attributes in Product_IDs, Product_Names and Quantities.

2. FUNCTIONAL DEPENDENCIES

```
Sale_ID -> Sale_Date, Customer_ID, Salesperson_ID, Payment_Mode
Customer_ID -> Customer_Name, Customer_City
Salesperson_ID -> Salesperson_Name
Product_ID -> Product_Name, Product_Category, Unit_Price
(Sale_ID, Product_ID) -> Quantity
Candidate key in 1NF: (Sale_ID, Product_ID)
```

3. CONVERT UNF TO 1NF

```
CREATE TABLE Sales_1NF (
 Sale_ID INT, Sale_Date DATE, Customer_ID INT,
 Customer_Name VARCHAR(50), Customer_City VARCHAR(50),
 Salesperson_ID INT, Salesperson_Name VARCHAR(50),
 Product_ID VARCHAR(10), Product_Name VARCHAR(100),
 Product_Category VARCHAR(50), Unit_Price DECIMAL(10,2),
 Quantity INT, Payment_Mode VARCHAR(20),
 PRIMARY KEY (Sale_ID, Product_ID)
);
INSERT INTO Sales_1NF VALUES
(1001,'2026-10-01',101,'Aarav Sharma','Delhi',501,'Neha Gupta','P01','Laptop','Electronics',60000,1,
'UPI'),
(1001,'2026-10-01',101,'Aarav Sharma','Delhi',501,'Neha Gupta','P02','Mouse','Accessories',800,2,'UPI'),
(1002,'2026-10-02',102,'Riya Verma','Lucknow',502,'Rahul Singh','P02','Mouse','Accessories',800,1,
'CARD'),
(1002,'2026-10-02',102,'Riya Verma','Lucknow',502,'Rahul Singh','P03','Keyboard','Accessories',1500,1,
'CARD'),
(1003,'2026-10-03',103,'Karan Mehta','Kanpur',501,'Neha Gupta','P01','Laptop','Electronics',60000,1,
'CASH'),
(1003,'2026-10-03',103,'Karan Mehta','Kanpur',501,'Neha Gupta','P04','Headphones','Accessories',2500,1,
'CASH');
SELECT * FROM Sales_1NF ORDER BY Sale_ID, Product_ID;
```

OUTPUT: 1NF RECORDS

Sale_ID	Product_ID	Product_Name	Category	Unit_Price	Qty	Customer	Payment
1001	P01	Laptop	Electronics	60000.00	1	Aarav Sharma	UPI
1001	P02	Mouse	Accessories	800.00	2	Aarav Sharma	UPI
1002	P02	Mouse	Accessories	800.00	1	Riya Verma	CARD
1002	P03	Keyboard	Accessories	1500.00	1	Riya Verma	CARD
1003	P01	Laptop	Electronics	60000.00	1	Karan Mehta	CASH
1003	P04	Headphones	Accessories	2500.00	1	Karan Mehta	CASH

Result: Each product is stored as an atomic value; the composite key is (Sale_ID, Product_ID).

4. CONVERT 1NF TO 2NF

```
CREATE TABLE Sale_2NF (
 Sale_ID INT PRIMARY KEY, Sale_Date DATE, Customer_ID INT,
 Customer_Name VARCHAR(50), Customer_City VARCHAR(50),
 Salesperson_ID INT, Salesperson_Name VARCHAR(50), Payment_Mode VARCHAR(20)
);
CREATE TABLE Sale_Item_2NF (
 Sale_ID INT, Product_ID VARCHAR(10), Product_Name VARCHAR(100),
 Product_Category VARCHAR(50), Unit_Price DECIMAL(10,2), Quantity INT,
 PRIMARY KEY (Sale_ID, Product_ID),
 FOREIGN KEY (Sale_ID) REFERENCES Sale_2NF(Sale_ID)
);
INSERT INTO Sale_2NF
SELECT DISTINCT Sale_ID, Sale_Date, Customer_ID, Customer_Name, Customer_City,
 Salesperson_ID, Salesperson_Name, Payment_Mode FROM Sales_1NF;
INSERT INTO Sale_Item_2NF
SELECT Sale_ID, Product_ID, Product_Name, Product_Category, Unit_Price, Quantity FROM Sales_1NF;
SELECT s.Sale_ID, s.Customer_Name, si.Product_ID, si.Product_Name, si.Quantity
FROM Sale_2NF s JOIN Sale_Item_2NF si ON s.Sale_ID = si.Sale_ID
ORDER BY s.Sale_ID, si.Product_ID;
```

OUTPUT: 2NF VERIFICATION

Sale_ID	Customer_Name	Product_ID	Product_Name	Quantity
1001	Aarav Sharma	P01	Laptop	1
1001	Aarav Sharma	P02	Mouse	2
1002	Riya Verma	P02	Mouse	1
1002	Riya Verma	P03	Keyboard	1
1003	Karan Mehta	P01	Laptop	1
1003	Karan Mehta	P04	Headphones	1

Result: Partial dependencies on Sale_ID and Product_ID are separated into sale and sale-item relations.

5. CONVERT 2NF TO 3NF

```
CREATE TABLE Customer (
 Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(50), Customer_City VARCHAR(50)
);
CREATE TABLE Salesperson (
 Salesperson_ID INT PRIMARY KEY, Salesperson_Name VARCHAR(50)
);
CREATE TABLE Product (
 Product_ID VARCHAR(10) PRIMARY KEY, Product_Name VARCHAR(100),
 Product_Category VARCHAR(50), Unit_Price DECIMAL(10,2)
);
CREATE TABLE Sale (
 Sale_ID INT PRIMARY KEY, Sale_Date DATE, Customer_ID INT, Salesperson_ID INT,
 Payment_Mode VARCHAR(20),
 FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
 FOREIGN KEY (Salesperson_ID) REFERENCES Salesperson(Salesperson_ID)
);
CREATE TABLE Sale_Item (
 Sale_ID INT, Product_ID VARCHAR(10), Quantity INT,
 PRIMARY KEY (Sale_ID, Product_ID),
 FOREIGN KEY (Sale_ID) REFERENCES Sale(Sale_ID),
 FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
);
INSERT INTO Customer SELECT DISTINCT Customer_ID, Customer_Name, Customer_City FROM Sales_1NF;
INSERT INTO Salesperson SELECT DISTINCT Salesperson_ID, Salesperson_Name FROM Sales_1NF;
INSERT INTO Product SELECT DISTINCT Product_ID, Product_Name, Product_Category, Unit_Price FROM
Sales_1NF;
INSERT INTO Sale SELECT DISTINCT Sale_ID, Sale_Date, Customer_ID, Salesperson_ID, Payment_Mode FROM
Sales_1NF;
INSERT INTO Sale_Item SELECT Sale_ID, Product_ID, Quantity FROM Sales_1NF;
```

6. QUERY 3 - VERIFY 3NF

```
SELECT s.Sale_ID, s.Sale_Date, c.Customer_Name, sp.Salesperson_Name,
       p.Product_Name, si.Quantity, s.Payment_Mode
FROM Sale s
JOIN Customer c ON s.Customer_ID = c.Customer_ID
JOIN Salesperson sp ON s.Salesperson_ID = sp.Salesperson_ID
JOIN Sale_Item si ON s.Sale_ID = si.Sale_ID
JOIN Product p ON si.Product_ID = p.Product_ID
ORDER BY s.Sale_ID, p.Product_ID;
```

OUTPUT: 3NF VERIFICATION

Sale_ID	Sale_Date	Customer	Salesperson	Product	Qty	Payment_Mode
1001	2026-10-01	Aarav Sharma	Neha Gupta	Laptop	1	UPI
1001	2026-10-01	Aarav Sharma	Neha Gupta	Mouse	2	UPI
1002	2026-10-02	Riya Verma	Rahul Singh	Mouse	1	CARD
1002	2026-10-02	Riya Verma	Rahul Singh	Keyboard	1	CARD
1003	2026-10-03	Karan Mehta	Neha Gupta	Laptop	1	CASH
1003	2026-10-03	Karan Mehta	Neha Gupta	Headphones	1	CASH

7. BCNF VERIFICATION

BCNF condition: For every non-trivial functional dependency X -> Y, X must be a superkey. In the final decomposition, each determinant is a candidate key/superkey of its respective relation. Therefore, the decomposed relations are in BCNF.

```
SHOW CREATE TABLE Customer;
SHOW CREATE TABLE Salesperson;
SHOW CREATE TABLE Product;
SHOW CREATE TABLE Sale;
SHOW CREATE TABLE Sale_Item;
```

OUTPUT: FINAL RELATION KEYS

Relation	Primary key	Foreign keys
Customer	Customer_ID	-
Salesperson	Salesperson_ID	-
Product	Product_ID	-
Sale	Sale_ID	Customer_ID; Salesperson_ID
Sale_Item	(Sale_ID, Product_ID)	Sale_ID; Product_ID

Dependencies: Customer_ID -> Customer_Name, Customer_City; Salesperson_ID -> Salesperson_Name; Product_ID -> Product_Name, Product_Category, Unit_Price; Sale_ID -> Sale_Date, Customer_ID, Salesperson_ID, Payment_Mode; (Sale_ID, Product_ID) -> Quantity.

8. LOSSLESS JOIN VERIFICATION

```
SELECT
 (SELECT COUNT(*) FROM Sales_1NF) AS Original_Rows,
 (SELECT COUNT(*) FROM Sale s
  JOIN Customer c ON s.Customer_ID = c.Customer_ID
  JOIN Salesperson sp ON s.Salesperson_ID = sp.Salesperson_ID
  JOIN Sale_Item si ON s.Sale_ID = si.Sale_ID
  JOIN Product p ON si.Product_ID = p.Product_ID) AS Reconstructed_Rows;
```

OUTPUT

Original_Rows	Reconstructed_Rows
6	6

9. DEPENDENCY PRESERVATION VERIFICATION

```
SELECT 'Customer FD violation' AS Check_Name, COUNT(*) AS Violations
FROM (SELECT Customer_ID FROM Customer GROUP BY Customer_ID
 HAVING COUNT(DISTINCT Customer_Name)>1 OR COUNT(DISTINCT Customer_City)>1) x
UNION ALL
SELECT 'Salesperson FD violation', COUNT(*)
FROM (SELECT Salesperson_ID FROM Salesperson GROUP BY Salesperson_ID
 HAVING COUNT(DISTINCT Salesperson_Name)>1) x
UNION ALL
SELECT 'Product FD violation', COUNT(*)
FROM (SELECT Product_ID FROM Product GROUP BY Product_ID
 HAVING COUNT(DISTINCT Product_Name)>1 OR COUNT(DISTINCT Product_Category)>1
 OR COUNT(DISTINCT Unit_Price)>1) x
UNION ALL
SELECT 'Sale FD violation', COUNT(*)
FROM (SELECT Sale_ID FROM Sale GROUP BY Sale_ID HAVING COUNT(*)>1) x
UNION ALL
SELECT 'Sale_Item key violation', COUNT(*)
FROM (SELECT Sale_ID, Product_ID FROM Sale_Item
 GROUP BY Sale_ID, Product_ID HAVING COUNT(*)>1) x;
```

OUTPUT

Check_Name	Violations
Customer FD violation	0
Salesperson FD violation	0
Product FD violation	0
Sale FD violation	0
Sale_Item key violation	0

RESULT

The Sales relation is normalized step-by-step from UNF to BCNF. The reconstructed join returns all six 1NF rows, and the dependency-preservation checks report zero violations.